Refactoring a Personal Toolbox: Scripts versus Shell Functions

A principled split for accumulated helpers in dotfiles and ~/bin

Ronald 'Ryy' G. Thomas

2026-04-25

Table of contents

1 Introduction **2**
- 1.1 Motivations . . . 3
- 1.2 Objectives . . . 3

2 Background: function or script? **4**

3 Guiding principle **5**
- 3.1 A familiar analogy: R functions and Rscript files . . . 5
- 3.2 Scope, in four lines . . . 5
- 3.3 Why the inversion is expensive . . . 6
- 3.4 Why the inversion happens . . . 6
- 3.5 When the rule cuts the other way . . . 7
- 3.6 What the rule is not . . . 7

4 Symptoms of an unaligned toolbox **7**

5 A categorisation matrix **7**

6 A seven-phase refactor **10**
- 6.1 Phase 1: extract the largest function . . . 10
- 6.2 Phase 2: extract the small functions . . . 10
- 6.3 Phase 3: convert microscripts to aliases . . . 10
- 6.4 Phase 4: standardise the rest . . . 11
- 6.5 Phase 5 (optional): autoload remaining functions . . . 11
- 6.6 Phase 6: install a guardrail . . . 11
- 6.7 Phase 7: decommission . . . 11

7 Things to watch out for **13**

8 Daily workflow after the refactor **13**

9 What did we learn? **14**
9.1 Lessons learnt . . . 14
9.2 Limitations . . . 14
9.3 Opportunities for improvement . . . 15

10 Wrapping up **15**

11 See Also **15**

12 Reproducibility **16**

13 Feedback **16**

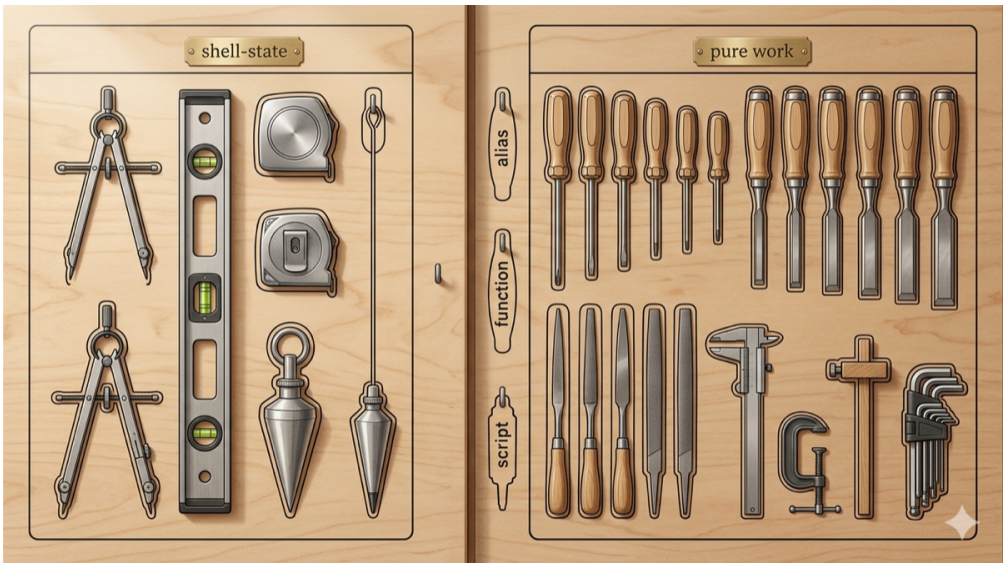

Figure 1: A workshop pegboard, organised so that each tool sits in its own labelled outline.

A personal toolbox earns its keep when each helper sits in the place that fits its job, with no doubt about which drawer to open.

1 Introduction

Most quantitative researchers carry a personal toolbox. It usually takes the form of a `~/bin` directory full of small shell scripts and a `.zshrc` (or `.bashrc`) file thick with custom aliases and functions. The contents accrue gradually: a one-liner that solved a particular irritation in 2019, a research-notes capture script written during a manuscript push in 2022, a git workflow function hammered out the week a co-author pushed a secret to GitHub.

Two years on, the toolbox works, but the boundary between 'script in `~/bin`' and 'function in `.zshrc`' has eroded. Some scripts in `~/bin` are aliases in disguise (a single `cd` plus `vim`). Some functions in `.zshrc` are two-hundred-line programs that have nothing to do with shell state. The toolbox still gets used, but it has become hard to reason about, hard to audit for security, and slow to load on shell start.

This post documents a principled refactor of such a toolbox. The worked example is biased toward a biostatistician's workflow (reproducible R analyses, Docker-based research compendia, frequent git commits, occasional HPC submission), but the rule that drives the refactor applies to any Unix-flavoured personal workspace.

A companion plan that operationalises the rule, with a concrete phase sequence and effort estimates, is referenced under See Also.

1.1 Motivations

- To establish a single rule that decides, for any helper, whether it belongs in `~/bin` or in shell config.
- To remove from `.zshrc` the logic that has no business mutating the current shell, lowering startup time and improving auditability.
- To bring versioned scripts under a linter (`shellcheck`) so that legacy quoting bugs surface before they bite.
- To eliminate microscripts that exist purely because the author did not realise the shell already has a primitive for them (the alias).
- To make the toolbox legible to future-self and to collaborators who inherit it during onboarding or Sabbatical handoff.

1.2 Objectives

1. State a guiding principle that resolves the function-versus-script question in every encountered case.
2. Provide a categorisation matrix so that an existing toolbox can be triaged in a single sitting.
3. Sequence the refactor into seven small phases, each independently shippable and reversible.
4. Document the gotchas that only surface when scripts are extracted out of a long-lived shell environment.

Errors and better approaches are welcome; see the Feedback section at the end.

Figure 2: A second monitor displaying terminal panes during an active refactor session.

2 Background: function or script?

A shell function is a named block of code that runs in the *current* shell process. It can read and modify the calling environment: change the working directory, export variables, edit the history, manipulate the dirstack, set or unset aliases. Functions live wherever they are sourced from, most commonly the interactive shell startup file.

A shell script is a file with execute permission that runs in a *new* process spawned from the current shell. It cannot, by design, mutate the calling shell's state. It receives arguments, environment, and file descriptors at launch; it returns an exit code; anything it prints flows back to the parent only via standard streams.

These two categories are not interchangeable. Some helpers genuinely require the function form (for example, a wrapper around `cd` that must affect the user's shell). Others fit equally in either form, but the choice has consequences for testability, auditability, startup performance, and the reader's experience.

A toolbox that has drifted out of alignment shows two kinds of inversion: scripts that should have been aliases, and functions that should have been scripts. The next section defines the rule that resolves both.

3 Guiding principle

The rule is short:

> A helper should be a shell function only when it must mutate the calling shell. Otherwise it should be a versioned, executable script in `~/bin`.

The remainder of this section unpacks the rule for an audience used to thinking carefully about scope, reproducibility, and the boundaries between processes: namely, computer-literate biostatisticians.

3.1 A familiar analogy: R functions and Rscript files

R users have lived with the same boundary for years without naming it. A pure helper that takes inputs, returns a value, and has no side effects is written as a function, often grouped with related functions in an R package. A one-shot batch job that loads a dataset, fits a model, writes a `.rds`, and emits a PDF is written as a script, run with `Rscript` or knit through Quarto. Nobody seriously proposes to put the entire batch pipeline inside `.Rprofile`, even though `.Rprofile` is technically capable of executing it. The pipeline does not need access to the interactive session's namespace, and burying it there would slow every R session to a crawl, hide it from version control review, and prevent any tooling from analysing it as a unit.

The shell case is the same boundary at a different scale. A shell function is the equivalent of a function loaded into the user's current R session: it can see and change that session's globals. A shell script is the equivalent of `Rscript` invoked from the command line: it gets fresh state, returns through stdout and an exit code, and finishes. Conflating the two in shell-land has the same costs as it would have in R-land, except that shell-land has historically been more tolerant of the conflation.

3.2 Scope, in four lines

Concretely, a helper *must* be a function when it does any of the following in a way that should outlast the helper's run:

- Change the working directory.
- Export, set, or unset shell variables.
- Modify aliases, options, key bindings, completion definitions.
- Manipulate the directory stack, history, or job table.

Anything outside that list (running a program, reading or writing files, formatting output, fetching from a URL, calling git, calling make, calling a database, parsing JSON) is a script's job, full stop. The rule scales: a function that *both* runs a long pipeline *and* needs to `cd` at the end is two helpers, one calling the other.

3.3 Why the inversion is expensive

A long shell function in `.zshrc` carries five hidden costs that biostatisticians, with their habits around `renv`, package versioning, and unit testing, will recognise immediately.

Startup latency. A function in `.zshrc` is parsed every time the shell starts. A script is parsed only when invoked. A two-hundred- line zsh function that the user calls a few times a day is re-parsed dozens of times for each one execution. The latency cost is the same shape as `library()` calls in `.Rprofile`: convenient on the surface, expensive in aggregate.

Lint and static analysis. `shellcheck` reads scripts; it does not introspect functions buried in dotfiles. Quoting bugs, unset variables, and incorrect test predicates that would be caught immediately in a script silently survive in a `.zshrc` function. The R parallel is `lintr` against package code versus `lintr` against ad-hoc chunks in `.Rprofile`.

Version control granularity. A script is a file. Its diffs are local, its history is local, blame is meaningful, and its commits do not interact with unrelated config changes. A function in a shared `.zshrc` competes with `PATH` exports, plugin lines, keybindings, and PROMPT changes for git history attention. Any non-trivial helper deserves its own git history, the same way a non-trivial R helper deserves its own `R/foo.R`.

Testability. A script can be invoked under a controlled environment with arguments, redirected I/O, and a captured exit code. The same logic embedded in a `.zshrc` function can only be tested by sourcing the dotfile and calling the function in a live shell, which is roughly as defensible as testing R code by copy-pasting it into the console.

Reuse across hosts and contexts. Scripts in `~/bin` are picked up by anything that respects `PATH`: cron, launchd, Make rules, Docker containers that mount the home directory, HPC submission scripts, scheduled GitHub Actions runners that source the toolbox. Functions are visible only to interactive zsh sessions. A research-backup helper hidden in `.zshrc` cannot be wired to a launchd job; the same logic in `~/bin` can be scheduled in seconds. This boundary maps almost exactly to the line between 'helpers exported by a package' and 'helpers defined inline in a notebook' in R work.

3.4 Why the inversion happens

A second pattern to notice: the inversions are not random. Helpers that grow up as one-liners (`cd somewhere && open something`) are written as scripts because the author did not yet know about aliases. Helpers that grow up gradually (a git workflow that started as a snippet, then accreted secret-scanning, then commit- message templating, then a confirmation prompt) end up as functions because they were edited each time a new shell was open and `.zshrc` was already in the editor. The forces driving each inversion are completely different, but the symptom (a toolbox where category does not predict location) is identical.

Recognising both forces matters because the corrective for each is different. The microscript that should have been an alias becomes an alias and disappears. The accumulated function that should have been a script becomes a script and gets the full hardening treatment: shebang, `set -euo pipefail`, quoted variables, a `-h`/`--help` flag, a `shellcheck` pass, and a place in version history.

3.5 When the rule cuts the other way

For completeness, the rule does sometimes route a current script *into* the function form. A helper that the author scripted but that genuinely needs to leave the user in a different directory is mis-categorised as a script. The give-away symptom: 'why does my shell not stay in the project directory after I run this?'. That helper's logic should move to a function, or, more cleanly, the script should be retained for non-interactive callers (cron, make) and a thin shell function should call it and apply the final `cd`.

3.6 What the rule is not

The rule is not 'all logic must leave `.zshrc`'. The dirstack helpers, the `ff` fzf-then-cd shortcut, the navigation jumps to project subdirectories: all are legitimately functions, because they exist precisely to mutate the calling shell. The point is to keep the function tier narrow and obviously appropriate, so that when a future reader sees a function in `.zshrc`, the existence of that function is itself evidence that it had to be one.

4 Symptoms of an unaligned toolbox

Three signs reliably indicate that a toolbox needs a refactor along these lines.

The shell startup file has crossed a thousand lines. Most of the bulk is unlikely to be configuration. It will be logic that should have been extracted long ago.

Microscripts in `~/bin` outnumber non-trivial scripts. A directory with thirty entries, half of which are under five lines, is mostly aliases waiting to be promoted.

At least one shell function imports a non-trivial external program. A `.zshrc` function that calls `gitleaks`, `aws`, `pandoc`, or `jq` is almost certainly mis-located. External programs imply a real workflow, and real workflows belong in versioned files.

5 A categorisation matrix

The matrix below sorts every helper a typical toolbox will contain into one of five fates. Triaging an existing toolbox is a single-sitting exercise: read each helper, ask the four questions in the principle, and place it.

Fate	Trigger
Keep as function	Helper must `cd`, `export`, modify aliases or history
Move from function to script	Function does no shell-state work; it just runs a program
Convert script to alias	Script is a single command line with no real argument parsing
Keep as script (harden)	Script is correctly placed but lacks shebang, quoting, lint
Decommission	Helper duplicates another, is referenced nowhere, or is dead

The first row is the smallest in any toolbox the author has inspected, typically four to twenty helpers. The remaining rows absorb everything else.

Figure 3: Editor showing a shell function being lifted into its own file under '~/bin'.

6 A seven-phase refactor

Each phase below is independently shippable and reversible. The ordering is by leverage: the early phases produce most of the visible improvement.

6.1 Phase 1: extract the largest function

In every toolbox the author has examined, a single function dominates by line count. Often it is a custom git commit workflow or a research-notes capture pipeline that grew over the years. That function alone typically contains 30 to 70 percent of the non-config bulk in `.zshrc`.

The extraction is mechanical:

- Create a new file in `~/bin` named after the helper. Mark it executable.
- Add a shebang that matches the syntax in use. Functions written with zsh-only constructs (`${(f)...}`, `${array:#}`, `setopt` semantics) require `#!/usr/bin/env zsh`; do not assume bash will parse them.
- Apply standard hardening: `set -u` always, `set -e` if the function does not deliberately tolerate failure, `set -o pipefail` if any command piping is involved.
- Decide on helpers. Sub-functions called only by the extracted helper can move with it (inline, or in a `lib/` directory the helper sources). External helpers used by other functions stay in shell config.
- Delete the original function from the shell startup file.
- Verify by exercising the helper end-to-end on a scratch workspace, including any failure paths.

This phase alone is usually responsible for a measurable shell startup speedup and for moving the largest single block of code under linting.

6.2 Phase 2: extract the small functions

After the large extraction, several smaller functions remain that also fail the principle: a Mathematica wrapper, a fuzzy file finder that calls vim, a make invocation that runs from any subdirectory. Each is a one-file extraction at most. Time per helper is small; the cumulative effect is roughly another fifty to eighty lines out of `.zshrc`.

6.3 Phase 3: convert microscripts to aliases

Inspect every script under fifteen lines in `~/bin`. Any whose body reduces to a single command pipeline (with no flag parsing, no branching, no output processing) is more honestly expressed as an alias. Examples that look exactly like aliases when written inline:

```
nohup /Applications/Ghostty.app/Contents/MacOS/ghostty \
  >/dev/null 2>&1 &
```

Promote each such script to an alias in shell config and delete the file. The benefit is twofold: the helper now appears in `alias` output (so it is discoverable by listing aliases), and the toolbox shrinks by one file per promotion.

6.4 Phase 4: standardise the rest

The remaining scripts in `~/bin` are correctly scripts. They typically need light hardening:

- Add a shebang, `#!/usr/bin/env bash` for bash-portable scripts.
- Add `set -euo pipefail` unless the script explicitly chooses a more permissive policy and documents why.
- Replace backticks with `$(...)`.
- Quote variable expansions: `"$1"`, `"$PWD"`, and so on.
- Resolve `shellcheck` warnings, or annotate with a `disable` comment that names a reason.
- Add a `-h`/`--help` flag for any script over thirty lines.
- Decide an extension policy and apply it consistently; standard Unix practice is to drop `.sh` from executables and reserve the extension for files that are explicitly sourced. Confirm that any scheduler entries (launchd, systemd, cron) referring to the script by name are updated together.

This is the long-tail phase. It can be done one helper at a time, amortised over normal work.

6.5 Phase 5 (optional): autoload remaining functions

For shell startup files that still feel heavy after phases 1 to 4, move the remaining functions out of the startup file into a function directory and `autoload` them:

```
fpath=(~/.zsh/functions $fpath)
autoload -Uz d ff za zw zy zf zt zs zp zr z0 zm ze zo zc zg
```

Each function lives in its own file under the function path. They load on first call, not on shell start. Skip this phase if the remaining functions are few or if having their bodies visible in the startup file is more valuable than the small startup cost.

6.6 Phase 6: install a guardrail

Add a `shellcheck` pre-commit hook scoped to `~/bin/*`, excluding binaries, R scripts, Python scripts, and any `archive/` directory. The point is not to catch a flood of issues at install time (phase 4 already handled those) but to prevent regressions as the toolbox continues to evolve. A single Make target, `make lint`, that runs `shellcheck` over every executable is sufficient.

6.7 Phase 7: decommission

Triage each remaining helper for redundancy. Common findings: two clipboard-to-notes helpers that do roughly the same thing under different names, a PDF viewer launcher that exists in two versions because the author once wanted to try a different viewer, an installer left behind from a tool that was removed years ago. Pick one of each, retire the rest. Git history preserves the removed code; the working tree should not.

Figure 4: A clean desk after the refactor; the same tools are present, but each sits where it belongs.

7 Things to watch out for

These pitfalls are easy to underestimate before starting and easy to recognise once seen.

1. **Zsh-only constructs.** Many shell functions accumulate zsh conveniences that bash will not parse: `${(f)var}` for newline splitting, `(s::)` for string-splitting flags, parameter modifiers like `${var:#}`. When extracting such a function to a script, the shebang must match the syntax. A bash shebang on a zsh function will fail in unhelpful ways.
2. **Schedulers refer to file names.** If `~/bin/foo.sh` is referenced by a launchd plist, a systemd unit, a cron entry, or a Makefile, renaming to `~/bin/foo` will silently break the scheduler. Search for every occurrence before renaming, and land both changes in the same commit.
3. **`PATH` ordering.** A script extracted from a function inherits `PATH` from the calling shell, which is usually fine. The exception is when the function previously ran inside a block that prepended a directory to `PATH`. The new script needs to do its own `PATH` adjustment or use absolute paths.
4. **Working directory at invocation.** A function inherits the caller's `$PWD`. A script does too, but if the script was previously a function that called itself recursively or that `cd`'d for its own purposes, that behaviour is now subprocess-local and may surprise the parent.
5. **History and dirstack contamination.** A function that previously ran `pushd`/`popd` to navigate around silently lost that capability when extracted to a script. If the helper genuinely needed the navigation, the helper was mis-classified and the rule routes it back to the function tier.
6. **`shellcheck` false positives.** External programs called with computed arguments (`"${args[@]}"`) sometimes trigger `SC2068`, `SC2086`, or `SC2154`. Disable per line with a comment that names the reason; do not blanket-disable the warning across the project.
7. **Secrets in extracted code.** A function previously sourced from `.zshrc` may have benefited from environment variables set elsewhere in the same file. The extracted script does not have that benefit. Either source the relevant env file explicitly inside the script, or read secrets via a tool such as `pass` or a credential helper.

8 Daily workflow after the refactor

The refactor pays back every day in three ways.

Discoverability. A new helper added to `~/bin` shows up under tab completion against `$PATH`. A new function added to the shell config shows up only in the current session and only after a reload.

Auditability. Every helper in `~/bin` is one `git log` away from full history; one `shellcheck` away from lint; one `cat` away from a complete reading. A helper buried in a thousand-line startup file requires the reader to first locate it.

Composability. Scripts in `~/bin` can be called from anything that respects `PATH`: Make rules, scheduled jobs, Docker containers mounted with the home directory, HPC submission scripts, peer collaborators who cloned the dotfiles repository. Functions can only be called from interactive shells

that have sourced them. The composability gain is the most important long-term return on the refactor.

9 What did we learn?

9.1 Lessons learnt

Conceptual:

- The function-versus-script question reduces to a single test: does the helper need to mutate the calling shell? Almost every other consideration follows from that one decision.
- The forces that produce inversions in each direction are different. Microscripts grow up because the author did not know about aliases; oversized functions grow up because the startup file is the file that happens to be open.
- A mature toolbox is mostly scripts, a small set of legitimate functions, and a layer of aliases. Any other distribution is evidence of accumulated misalignment.

Technical:

- A long zsh function may not be portable to bash, even when the prose-level intent looks generic. Confirm the shebang against the syntactic features actually in use.
- `shellcheck` is the highest-leverage tool to apply at the end of an extraction. It catches more than half of the latent quoting and unset-variable bugs in legacy scripts.
- Naming and extension conventions only need to be applied consistently within a toolbox. The choice of `.sh` versus no extension matters less than the absence of a third option appearing for no reason.

Gotchas:

- Renaming a script that a scheduler refers to silently breaks the schedule. Always grep for the file name across `launchd/`, `systemd/`, `Makefile`, and any related repositories before renaming.
- Functions that depend on shell-specific syntax cannot be blindly retitled with a bash shebang. Prefer the zsh shebang unless the syntax has been explicitly portabilised.

9.2 Limitations

The refactor described here addresses helpers in shell-land specifically. It does not:

- Address Python, R, or other-language helpers whose execution model differs from shell scripts.
- Replace a real configuration management system. Tools such as Ansible, Nix, or chezmoi solve a larger problem and may be appropriate when the personal toolbox grows past a few dozen files or needs to be deployed to multiple machines.
- Provide automated migration. Each helper still needs a human to read it, classify it, and verify the extraction.
- Compose with shell frameworks that load logic from many files (Oh My Zsh, prezto). Those frameworks have their own conventions; the rule still applies, but the implementation details change.

9.3 Opportunities for improvement

1. Encode the rule as a small linter that scans `.zshrc` for functions and flags any whose body never references shell state. This would surface candidates for extraction over time rather than at one big-bang refactor.
2. Add a CI workflow that runs `shellcheck` over `~/bin` on every push to the dotfiles repository.
3. Generate `~/bin/nav` (a navigation cheat sheet) from the `autoload`ed function metadata, so adding a new shortcut automatically updates the help output.
4. Build a thin packaging layer around shared helpers so they can be installed on collaborator machines without copying files by hand. A simple Make target plus a manifest is usually sufficient.
5. Track helper invocation frequency via the shell's `precmd` hook for one week per year, and use the result to prune helpers nobody actually calls.

10 Wrapping up

A personal toolbox is software, even when it is small, even when it is private, even when it grew organically. Treating it as software (with a rule for how it is structured, lint applied to the parts that should be linted, version control applied to the parts that change, and a maintenance pass when the bulk crosses some threshold) pays back every day for the next decade.

The rule that drives the refactor is short enough to remember: function only when shell state must change, script otherwise. The work to apply it is finite, ordered, and reversible. A single afternoon, divided across the seven phases above, is enough to move a toolbox from accumulated drift back into alignment.

Main takeaways:

- The rule reduces every helper-placement decision to a single question about shell state.
- The largest single win is extracting the dominant function out of the shell startup file and into `~/bin`.
- Microscripts that read like one-liners belong as aliases, not files.
- A `shellcheck` guardrail prevents the same drift from re-accumulating.

11 See Also

Related posts:

- Setting up dotfiles on GitHub: versioning the shell startup file alongside the rest of the configuration.
- A research backup system: a worked example of a helper that belongs as a script invoked from a scheduler, not as a function in a shell.

Key resources:

- shellcheck: static analyser for shell scripts.

- zsh manual: parameter expansion: authoritative reference for zsh-only constructs encountered during extraction.
- Bash hackers wiki: portable shell: inventory of constructs that fail to port between shells.
- The twelve-factor app: the configuration discipline that the rule echoes for shell environments.

---

12 Reproducibility

Tested configuration:

Component	Version
Operating system	macOS 15.4
Shell	zsh 5.9
shellcheck	0.10.0
Last verified	2026-04-25

Configuration files:

- `analysis/configs/refactor_plan.md`: the seven-phase plan in the format used to drive the worked refactor.

---

13 Feedback

Corrections, suggestions, and questions are welcome. Please open an issue or pull request on the GitHub repository or send an email to user@example.com.